Track-System – Backend Files Map

Last updated: 17 Oct 2026 | Covers everything up to step 6 (forgot password by email: reset links, email sending, the reset page, tests) | What each file is for and how they are wired together | **[NEW]** = added in step 6

1. Where everything sits

```
Track-System/
  Database/                        <- [NEW] SQL scripts, run by hand in SSMS (in git)
    FinalProject_schema.sql        <- [NEW] the baseline: all 10 tables, no data
    seed_test_employee.sql         <- [NEW] DEV ONLY: one test employee (password Test1234!)
    changes/                       <- [NEW] every later DB change, numbered, run once on each PC
      001_create_PasswordResetTokens.sql  <- [NEW] the reset-links table
  Frontend/ (React + Vite)
    .env.development               <- VITE_API_URL (NOT in git: create it on each PC)
    src/
      main.jsx                     <- starts the session keeper once (startSession)
      App.jsx                      <- the routes ([NEW] + /reset-password)
      api/
         client.js                 <- shared axios: API address, adds the token, 401 -> Login
         session.js                <- refreshes the token while active, idle logout
      utils/
         passwordRules.js          <- the same password rules as the API
      pages/Login, ChangePassword <- call the API
      pages/ForgotPassword         <- [NEW] asks for the eNum and calls the API
      pages/ResetPassword          <- [NEW] the page the email link opens
  Backend/
    TrackSystem.Api/
      TrackSystem.Api.csproj       <- list of libraries + UserSecretsId ([NEW] + MailKit)
      appsettings.json             <- settings (DB, Jwt, Cors, RateLimit, [NEW] Email, PasswordReset)
      appsettings.Testing.json              <- test settings (FinalProject_Test, limit 1000, [NEW] no wait)
      appsettings.RateLimitTesting.json     <- settings of the 429 test (limit 3)
      Program.cs                   <- the server: setup + endpoints
      Data/
         AppDbContext.cs           <- the bridge to the database
      Models/                      <- C# classes that mirror tables
         Employee.cs               <- one row of dbo.Employees
         LoginLog.cs               <- one row of dbo.LoginLogs
         PasswordResetToken.cs     <- [NEW] one row of dbo.PasswordResetTokens
      Dtos/                        <- shapes of data sent by the browser
         LoginRequest.cs           <- the login body (eNum + password)
         ChangePasswordRequest.cs <- the change-password body
         ForgotPasswordRequest.cs <- [NEW] the forgot-password body (eNum)
         ResetPasswordRequest.cs  <- [NEW] the reset-password body (token + new password)
      Security/
         PasswordRules.cs          <- the rules for a new password
         ResetTokens.cs            <- [NEW] makes a reset token and its SHA-256 hash
      Services/                    <- [NEW]
         IEmailSender.cs           <- [NEW] the contract "something that can send an email"
         SmtpEmailSender.cs        <- [NEW] the real sender (MailKit -> Mailpit or Gmail)
      login.http                   <- local test requests (ignored by git)
      Docs/
         Backend-Files-Map.pdf     <- this file
         Backend-Glossary.pdf      <- the words to learn
      bin/ obj/ (build output - ignored by git)
    TrackSystem.Api.Tests/         <- the automatic tests (xUnit)
      TrackSystem.Api.Tests.csproj <- test libraries + reference to TrackSystem.Api
      Security/
         PasswordRulesTests.cs     <- unit tests (no API, no DB)
         ResetTokensTests.cs       <- [NEW] unit tests for ResetTokens
      Infrastructure/              <- test helpers, not tests
         TestApiFactory.cs         <- starts the API in memory + builds the test DB ([NEW] + fake mailbox)
         ApiCollection.cs          <- one shared test API for all Api/ tests
         RateLimitApiFactory.cs    <- a 2nd test API with limit 3
         FakeEmailSender.cs        <- [NEW] test mailbox: remembers emails, sends nothing
      Api/                         <- integration tests, one file per area
         HealthTests.cs LoginTests.cs TokenTests.cs EmployeesTests.cs
         ChangePasswordTests.cs RefreshTests.cs RateLimitTests.cs
         ForgotPasswordTests.cs ResetPasswordTests.cs   <- [NEW]
      bin/ obj/ (build output - ignored by git)
  .gitignore (what git must not upload)

Outside the repo, on each PC:
  C:\Users\<you>\AppData\Roaming\Microsoft\UserSecrets\<id>\secrets.json
                                 <- the JWT signing key (Jwt:Key) [NEW] + optional Email:* for real Gmail
  [NEW] Mailpit (installed with winget)
```

```
                                <- fake mail server: SMTP on port 1025, inbox at http://localhost:8025
SQL Server (localhost\SQLEXPRESS):
  FinalProject                  <- the real data ([NEW] + table PasswordResetTokens)
  FinalProject_Test             <- made by dotnet test, dropped and rebuilt on every run
```

2. What each file does

File	What it is	What it does	When do I touch it
[NEW] Database/FinalProject_schema.sql	SQL script (baseline)	Creates all 10 tables of FinalProject (keys, foreign keys, defaults), schema only, no data. Generated from SSMS. It shows the DB as it was before the first change script.	Never edited: later changes go to Database/changes/
[NEW] Database/seed_test_employee.sql	SQL script (dev only)	Adds one test employee (active, first login, password Test1234! stored as a BCrypt hash). Safe to run twice (IF NOT EXISTS). Never on a real DB.	When a new PC needs a test user
[NEW] Database/changes/001_create_PasswordResetTokens.sql	SQL change script	Creates dbo.PasswordResetTokens: tokenID, eNum (FK -> Employees, ON DELETE CASCADE), tokenHash char(64) UNIQUE, expiresAt, usedAt, createdAt. The number says the order the scripts must run in.	Never edited after it ran; a new change = a new file 002_...
TrackSystem.Api.csproj	Project definition	Lists the .NET version and every library: EF Core (SQL Server), BCrypt.Net-Next, JwtBearer, System.Security.Cryptography.Pkcs 10.0.13 (replaces a vulnerable indirect version), **[NEW]** MailKit (sends email; brings MimeKit). Plus the UserSecretsId (the name of the secrets folder, not a secret).	Every time we add a library, or a vulnerability warning appears
appsettings.json	Settings file (not code)	Connection string, Jwt (Issuer, Audience, ExpiresMinutes 30), Cors:AllowedOrigins, RateLimit:AuthPermitLimit. **[NEW]** Email: SmtpHost, SmtpPort, UseStartTls, FromAddress, FromName (Mailpit by default). **[NEW]** PasswordReset: TokenMinutes 30, ResetPageUrl, MinResponseMs 1500. No key or password is in this file.	When a server, DB, URL or limit setting changes
appsettings.Testing.json	Test settings	Loaded on top of appsettings.json only in the environment "Testing" (the shared test API). FinalProject_Test, limit 1000, **[NEW]** PasswordReset:MinResponseMs 0 (no 1.5 s wait in tests). In git.	When the test DB or a test setting changes
appsettings.RateLimitTesting.json	Test settings	Same idea for the environment "RateLimitTesting": FinalProject_Test and a limit of 3. Used only by the 429 test.	Rarely
secrets.json (User Secrets)	Secret settings, outside the repo	Holds Jwt:Key, the key that signs and checks every token. Each PC has its own. Loaded only in Development; the tests make their own random key. **[NEW]** Optional: Email:SmtpHost / Port / UseStartTls / FromAddress / UserName / Password to send real mail through Gmail (App Password) on this PC only.	Only with the dotnet user-secrets commands
Data/AppDbContext.cs	The bridge to the DB	Represents the database in C#. One DbSet line per table: Employees, LoginLogs, **[NEW]** PasswordResetTokens. The tests also build FinalProject_Test from these models.	Every time we add a table
Models/Employee.cs	A model (a shape)	One row of dbo.Employees. The Password property holds a BCrypt hash, never the real password.	When a column in dbo.Employees changes
Models/LoginLog.cs	A model (a shape)	One row of dbo.LoginLogs: one login attempt (logID, eNum, loginDate, ipAddress, success).	When a column in dbo.LoginLogs changes
[NEW] Models/PasswordResetToken.cs	A model (a shape)	One row of dbo.PasswordResetTokens: one reset link. [Index] makes TokenHash unique, [Column(TypeName = "char(64)")] keeps its SQL type, and the navigation property Employee (with [ForeignKey]) gives the FK, also in the test DB.	When a column of that table changes
Dtos/LoginRequest.cs	A DTO (input shape)	What the browser sends to the login: eNum and password. ASP.NET turns the JSON body into this object.	If the login form sends other fields
Dtos/ChangePasswordRequest.cs	A DTO (input shape)	What the change-password form sends: CurrentPassword and NewPassword.	If the form sends other fields

File	What it is	What it does	When do I touch it
[NEW] Dtos/ForgotPasswordRequest.cs	A DTO (input shape)	What the Forgot Password page sends: only ENum. The email address is always taken from the DB.	If the form sends other fields
[NEW] Dtos/ResetPasswordRequest.cs	A DTO (input shape)	What the Reset Password page sends: Token (from the link) and NewPassword.	If the form sends other fields
Security/PasswordRules.cs	A static helper class	Validate(password) -> null if it passes, else the first broken rule as a message. Rules: 8-64 characters, one capital, one number, one special character. Used by change-password [NEW] and reset-password.	When a password rule changes (change passwordRules.js too)
[NEW] Security/ResetTokens.cs	A static helper class	Create() -> (Token, Hash): 32 secure random bytes as 64 lowercase hex chars, plus its SHA-256 hash. Hash(token) -> the same 64-char hash, used to find the row from a link.	Rarely
[NEW] Services/IEmailSender.cs	An interface (a contract)	One method: SendAsync(to, subject, htmlBody). Endpoints ask for this contract, so the real sender can be swapped (tests use FakeEmailSender).	If emails need more (e.g. attachments)
[NEW] Services/SmtpEmailSender.cs	The real email sender	Builds the email with MimeKit and sends it with MailKit: connect (StartTls or none), log in only if Email:UserName is set, send, disconnect. A missing setting -> a clear error message.	When the way of sending changes
Program.cs	The server	Setup area, then endpoints. Setup registers CORS, the two access rules, the rate limiter "auth", a dummy hash, [NEW] IEmailSender -> SmtpEmailSender (AddScoped) and reads TokenMinutes, ResetPageUrl, MinResponseMs. CreateToken builds every token. [NEW] New endpoints: forgot-password and reset-password.	Every new endpoint or new service
.gitignore	Git rules	Stops build files, secrets, *.http files and .env.* files from being uploaded.	Rarely
login.http	Local test file (REST Client)	Ready-made requests for login, employees, change-password, the RESET request, refresh, [NEW] FORGOT 1/2 and RESET-CHECK 1-3. Not in git (it holds a test password).	When testing endpoints by hand
Frontend/.env.development	Frontend settings	One line: VITE_API_URL=http://localhost:5163. Not in git, so each PC creates it by hand.	When the API address changes
src/api/client.js	Shared axios	baseURL from .env.development. Before each request: adds Authorization: Bearer <token> from sessionStorage. After a 401 (not from login): removes the token and goes to "/".	Rarely; every page imports it to call the API
src/api/session.js	Session keeper	On click / key / mouse / scroll: token older than 60 s -> POST /api/auth/refresh. Every 15 s: token expired -> removed, back to Login (idle logout).	When the session rules change
src/utils/passwordRules.js	Frontend rules	The same rules and messages as PasswordRules.cs, so the user sees the error before sending. Used by ChangePassword [NEW] and ResetPassword.	Together with PasswordRules.cs
src/main.jsx	Frontend start	Calls startSession() once, above createRoot.	Rarely
src/App.jsx	Frontend routes	Which page answers which address: /, /forgot-password, /change-password, [NEW] /reset-password.	Every new page
pages/Login, ChangePassword	React pages	Login: digits-only eNum, POST login, saves the token, mustChangePassword -> /change-password. ChangePassword: current / new / confirm, POST change-password, saves the new token.	When the page or its flow changes
[NEW] pages/ForgotPassword	React page	Employee number (digits only) -> POST forgot-password -> the server's green message. The button shows "Sending..." and is disabled during the ~1.5 s wait.	When the page or its flow changes
[NEW] pages/ResetPassword	React page (new)	Reads ?token= from the address (useSearchParams). New + confirm password with the same rules -> POST reset-password -> green message + "Go to Login". No token or a bad link -> red message + "Request a new link". Its .module.css is a copy of ChangePassword's.	When the page or its flow changes

File	What it is	What it does	When do I touch it
TrackSystem.Api.Tests.csproj	Test project definition	xUnit, the test runner, Microsoft.AspNetCore.Mvc.Testing, and a reference to TrackSystem.Api (so the tests see Program, AppDbContext, PasswordRules, **[NEW]** ResetTokens, IEmailSender).	When a test library is added
Security/PasswordRulesTests.cs	Unit tests	Calls PasswordRules.Validate directly: a valid password, each broken rule, the 8 / 64 / 65 limits, null and empty. No API, no DB. **[NEW]** Now in the Security/ folder.	When a password rule changes
[NEW] Security/ResetTokensTests.cs	Unit tests	Token = 64 lowercase hex chars; Create's hash = Hash(token); two tokens differ; Hash("abc") = the official SHA-256 test value.	When ResetTokens changes
Infrastructure/TestApiFactory.cs	Test helper (the factory)	Starts the API in memory in "Testing" with a random Jwt:Key; drops and recreates FinalProject_Test (refuses any other DB name). **[NEW]** Replaces IEmailSender with the Emails fake (ConfigureTestServices). Helpers: CreateEmployeeAsync, GetEmployeeAsync, GetLoginLogsAsync, LoginAsync, SetStatusAsync, **[NEW]** LastResetTokenFor, GetResetTokensAsync, ExpireResetTokensAsync.	When tests need a new helper
Infrastructure/ApiCollection.cs	Test helper	Collection "Api": one shared TestApiFactory for every class marked [Collection("Api")]. Those classes run one after another, never two at once on the test DB.	Rarely
Infrastructure/RateLimitApiFactory.cs	Test helper	A second in-memory API in "RateLimitTesting" (limit 3) with its own counters, so the 429 test does not affect the shared API. Its tests only call login, so it needs no fake mailbox.	Rarely
[NEW] Infrastructure/FakeEmailSender.cs	Test helper (test double)	Implements IEmailSender but sends nothing: every email goes into the list Sent (To, Subject, HtmlBody), so a test can read the reset link.	If IEmailSender changes
Api/*Tests.cs (9 files)	Integration tests	Real HTTP requests to the in-memory API, then checks of the status, the JSON body, the token claims and the DB rows. Health, Login, Token, Employees, ChangePassword, Refresh, RateLimit, **[NEW]** ForgotPassword (same answer, email only for active, only the hash saved, old link replaced), **[NEW]** ResetPassword (success + unlock, used / replaced / expired / bad / deactivated -> 400, weak password keeps the link).	When an endpoint changes or a new one is added

3. How the files are connected

```
Browser / React (port 5173)
   | src/api/client.js: baseURL from .env.development + "Authorization: Bearer <token>"
   | src/api/session.js: POST /api/auth/refresh while the user is active
   | [NEW] pages/ResetPassword: reads ?token=... from the email link
   v
Program.cs (port 5163)
   | UseCors: allows only the origin in ----------------> appsettings.json (Cors)
   | UseRateLimiter: login, change-password, [NEW] forgot + reset, per IP -> appsettings.json (RateLimit)
   | UseAuthentication: checks the token with the key from --> secrets.json
   |                    and Issuer / Audience from ---------> appsettings.json
   | UseAuthorization: default rule / "PasswordChange" -> 401 or 403
   |
   | the bodies arrive as -------------------> Dtos/*Request.cs
   | a new password is checked by ------------> Security/PasswordRules.cs
   | [NEW] reset links are made / hashed by ---> Security/ResetTokens.cs
   | [NEW] emails go through -----------------> Services/IEmailSender -> SmtpEmailSender (MailKit)
   |                                              -> Mailpit :1025 (or Gmail :587 via User Secrets)
   | Program.cs uses ------------------------> Data/AppDbContext.cs -> Models/*.cs
   v
AppDbContext --(EF Core turns C# into SQL)--> SQL Server, DB FinalProject

dotnet test:
TrackSystem.Api.Tests --(project reference)--> TrackSystem.Api
   TestApiFactory: starts Program.cs in memory, environment "Testing" -> appsettings.Testing.json
                   [NEW] IEmailSender replaced by FakeEmailSender (no real mail)
                   AppDbContext --> SQL Server, DB FinalProject_Test (dropped + created every run)
```

This file	needs / uses	Why
Program.cs	appsettings.json	Reads DefaultConnection, Jwt:Issuer, Jwt:Audience, Jwt:ExpiresMinutes, Cors:AllowedOrigins, RateLimit:AuthPermitLimit, **[NEW]** PasswordReset:TokenMinutes / ResetPageUrl / MinResponseMs.
Program.cs	secrets.json (User Secrets)	Reads Jwt:Key. The same key signs tokens and checks them on every request. Without it the server refuses to start.
Program.cs (CreateToken)	appsettings.json + secrets.json	One place that builds every token (login, change-password, refresh): sub, role, mustChangePassword, 30 min.
Program.cs (/api/auth/login)	Dtos/LoginRequest.cs	The login endpoint receives the body as a LoginRequest object.
Program.cs (/api/auth/login)	AppDbContext.Employees	Finds the employee, checks the hash with BCrypt, updates failedLoginAttempts / lockoutEnd. Locked or not active -> the same generic 401. Unknown eNum or locked -> a dummy BCrypt check (equal timing).
Program.cs (/api/auth/login)	AppDbContext.LoginLogs	Adds one row per attempt. The row and the employee changes are saved together with SaveChangesAsync.
Program.cs (/api/auth/change-password)	ChangePasswordRequest, PasswordRules, AppDbContext.Employees	eNum from the token's sub. Checks the current password, new != current, the rules; saves the new hash, IsFirstLogin = false, returns a new token.
Program.cs (/api/auth/refresh)	AppDbContext.Employees	Re-reads the employee: deleted or not active -> 401, else a fresh 30-minute token.
[NEW] Program.cs (/api/auth/forgot-password)	ForgotPasswordRequest, ResetTokens, AppDbContext, IEmailSender, ILogger	Active employee only: old unused links -> usedAt, new row with the token's hash, email with the link (a mail error is only logged). Every case waits until MinResponseMs and gives the same 200.
[NEW] Program.cs (/api/auth/reset-password)	ResetPasswordRequest, ResetTokens, PasswordRules, AppDbContext	Finds the row by Hash(token) with Include(Employee). Unknown / used / expired / not active -> 400. Rules -> 400. Else new BCrypt hash, IsFirstLogin false, unlock, usedAt = now, one SaveChangesAsync.
[NEW] SmtpEmailSender.cs	IConfiguration (appsettings.json + secrets.json)	Reads the Email:* settings on every send. User Secrets override appsettings.json, so one PC can use Gmail while git keeps Mailpit.
Program.cs (/api/employees)	AppDbContext.Employees	Reads the employees and picks only the safe fields. Runs only with a valid token whose mustChangePassword is "false".
AppDbContext.cs	Models/*.cs	Each DbSet line says which model shapes which table.
Models/LoginLog.cs	dbo.LoginLogs (the real table)	Property names must match the columns. eNum has no foreign key by design (unknown numbers are logged too).
[NEW] Models/PasswordResetToken.cs	dbo.PasswordResetTokens + 001_...sql	The script builds the real table; the model's attributes make EF build the same table in the test DB. Change both together.
src/utils/passwordRules.js	Security/PasswordRules.cs	Same rules, same messages: change both together.
src/api/session.js	src/api/client.js, /api/auth/refresh	Uses the shared axios, so a 401 from refresh also sends the user to Login.
[NEW] ResetPassword page	appsettings.json (ResetPageUrl), App.jsx	The email link is ResetPageUrl + ?token=..., so the route /reset-password must match it.
TrackSystem.Api.Tests	TrackSystem.Api (project reference)	The tests call the real classes and start the real Program.cs.
TestApiFactory.cs	appsettings.Testing.json, AppDbContext, **[NEW]** FakeEmailSender	Environment "Testing" picks the test settings; EnsureCreated builds FinalProject_Test from the models; the fake replaces the real sender.
Everything	TrackSystem.Api.csproj	If a library version does not match the .NET version, nothing builds.

4. The addresses that exist today (all inside Program.cs)

Address	Needs a token?	Answers	What it proves
GET /api/health	No	{"status":"ok"}	The server itself is alive
GET /api/db-check	No	{"database":"connected"}	The server reached the FinalProject database
POST /api/auth/login	No (this is where you get one). Rate limit: 10 per minute per IP	200: message, eNum, mustChangePassword, token (30 min). 401: "Invalid employee number or password", also when locked or not active. 429 after 10 tries in a minute.	BCrypt check, 5-try lockout, logging of every attempt, equal timing, token issued
GET /api/employees	Yes, the default rule	200: the list (no passwords). 401: no, fake, edited or expired token. 403: valid token with mustChangePassword "true".	The token guard and the default rule work
POST /api/auth/change-password	Yes, rule "PasswordChange" (also first-login tokens). Rate limit too	200: message, mustChangePassword false, a new token. 400 + message: wrong current password, same as current, or a broken rule. 401: bad token. 429.	Forced first-login change, password rules
POST /api/auth/refresh	Yes, rule "PasswordChange". No body, no rate limit	200: token, mustChangePassword (read from the DB). 401: no token, or the employee is deleted / not active.	Sliding 30-minute session
[NEW] POST /api/auth/forgot-password	No. Rate limit too. Body {eNum}	Always 200 "If the account exists, a reset link was sent to its email." after at least 1.5 s. Only an active employee gets an email. 429.	No user enumeration (same text, same time), one working link per employee
[NEW] POST /api/auth/reset-password	No (the link's token is the proof). Rate limit too. Body {token, newPassword}	200 "Your password has been reset. You can log in now." 400 "This link is invalid or has expired. Please request a new one." or a password-rule message (the link stays usable). 429.	One-time, 30-minute links; reset also unlocks; never 401 (so React does not log out)

5. Rules for this project

Rule	Why
Services are registered above var app = builder.Build();	After the server is built, nothing can be registered any more.
After Build(), in this order: UseCors, UseRateLimiter, UseAuthentication, UseAuthorization, then the endpoints	The request pipeline runs top to bottom: CORS headers, count the attempt, identify the caller, apply the rules.
Server and DB names live in appsettings.json. Secret keys go to User Secrets.	One place to change, and nothing secret reaches the public GitHub repo. **[NEW]** SMTP passwords (Gmail App Password) too.
All database traffic goes through AppDbContext	One single door to the DB.
The model may hold the password hash; an endpoint never sends it	Each endpoint chooses which fields leave the server. A test checks the /api/employees answer has no hash.
Never put secrets in a token	The token payload is readable by anyone. Only IDs and flags go in it.
Times used by code are UTC; times read by people are Israel time	lockoutEnd **[NEW]** and the reset-token times are UTC (no daylight-saving jumps). LoginLogs.loginDate is Israel time. Sort logs by logID.
401 = token problem, 400 = wrong input, 403 = valid token but not allowed here, 429 = too many attempts	React reacts differently: 401 logs out, 400 shows the message, 403 keeps the user logged in. **[NEW]** That is why a bad reset link is 400, not 401.

Rule	Why
Password rules live in two places: PasswordRules.cs and passwordRules.js	The browser checks first for a quick message, the API checks again for safety. Change both.
A DB structure change is run on both machines and written in the Drive docs	The DB is not in git; each PC has its own copy.
[NEW] Every DB change is a numbered script in Database/changes/ (001_, 002_ ...), run once on each PC, never edited after it ran	Both PCs get exactly the same tables, in the same order.
[NEW] Sending email goes through IEmailSender, never MailKit directly in an endpoint	Tests swap in FakeEmailSender, so no test ever sends real mail.
[NEW] Endpoints that reveal nothing about accounts answer the same text in the same time	forgot-password: same 200 for unknown / not active / real, padded to MinResponseMs.
Tests use only FinalProject_Test, never FinalProject	The test setup drops a whole database. TestApiFactory stops if the name is anything else.
Every test creates its own employee	Tests do not depend on each other's leftovers, and their order does not matter.
dotnet test runs inside Backend\TrackSystem.Api.Tests. All green before every commit	A red test means something that worked before is broken.
After a code or appsettings change: save, stop (Ctrl+C), run again	The server reads both only at startup.
Run dotnet commands for the API inside Backend\TrackSystem.Api	From the repo root they fail with MSB1003.
bin/, obj/, secrets, *.http and .env.* files stay out of git	Run git status before every commit.
Frontend/.env.development is not in git	Each PC creates it: VITE_API_URL=http://localhost:5163.
[NEW] Mailpit must be running to test forgot-password by hand	Otherwise sending fails: the API logs the error, the user still sees the same 200.

6. Open items

Item	Details
[NEW] Closed in step 6	Forgot password by email: PasswordResetTokens table, one-time 30-minute links (only the SHA-256 hash is stored), MailKit + Mailpit, IEmailSender, equal answer and time, ResetPassword page, 18 new tests (56 in total). DB changes are now tracked in git (Database/changes/).
[NEW] Partner PC	Run Database/changes/001_create_PasswordResetTokens.sql once, install Mailpit (winget install axllent.mailpit), then dotnet test = 56 green.
[NEW] Real email (Gmail)	Works without code changes: Email:* in User Secrets with a Gmail App Password. Test employees must have addresses you own. Production would use a real sending service.
[NEW] Old reset-token rows	Kept as history (used / replaced / expired). Later: a cleanup job or a limit of N requests per employee per hour (count the rows).
[NEW] Next: ERP activation	pending -> Activate -> password by email will reuse IEmailSender and the same one-time token idea.
Main page after login	Login, ChangePassword (and the reset flow's "Go to Login") still end without a main page; the console.log TODOs remain.
/api/employees: any logged-in user	Needs a valid token (and a changed password), but any role may call it. Later: role rules.
Old token after a password change	Stays valid until it expires (stateless JWT). Harmless after a first-login change: it still says "true", so it opens only change-password and refresh. **[NEW]** After a reset by email an older token (if any) also stays valid until it expires.
Shared server name	If a PC's SQL instance has another name, override DefaultConnection in User Secrets on that PC only. The test settings files use localhost\SQLEXPRESS too.
/api/db-check is open	A development tool; remove or protect it before any real deployment.

Item	Details
Files not created yet	Models for the other 8 tables, endpoint files (to keep Program.cs short), output DTOs, a Services layer for the login logic.